

MODULE 01 · PART 1 - FOUNDATIONS

Logical Query Processing Order

A query is not evaluated in the order you wrote it. Almost every surprise in this course starts there.

PART 1 of 8 MODULE 1 of 26 RUNS about 32 min

You write a query in one order. The engine
evaluates it in another.

Four questions, one answer

- ◆ Why can't WHERE see the alias I just defined in the SELECT list?
- ◆ Why can't WHERE filter a row number the same query just computed?
- ◆ Why does GROUP BY reject a column that is obviously determined by the key?
- ◆ Why did adding a filter delete rows from a LEFT JOIN that should have been preserved?

THE TAKEAWAY

Four questions that look unrelated. One cause, and one place to look. This module is that place.

Two orders

AS YOU WRITE IT

Written order

- 1 **SELECT**
- 2 FROM
- 3 WHERE
- 4 GROUP BY
- 5 HAVING
- 6 QUALIFY
- 7 ORDER BY
- 8 LIMIT

SELECT is written first.

AS IT IS EVALUATED

Logical order

- 1 FROM and JOIN
- 2 WHERE
- 3 GROUP BY
- 4 HAVING
- 5 WINDOW
- 6 QUALIFY
- 7 **SELECT**
- 8 DISTINCT
- 9 ORDER BY
- 10 LIMIT

SELECT is evaluated seventh.

THE TAKEAWAY

SELECT is written first and evaluated seventh. That single fact explains three of the four questions on the last slide.

What each step can see

STEP	WHAT IT CAN REFER TO
1. FROM and JOIN	Base tables and their columns. Nothing derived yet.
2. WHERE	Table columns only. No aggregates, no windows, no aliases.
3. GROUP BY	Table columns and expressions. The detail rows end here.
4. HAVING	Grouping keys and aggregates. Still no window results.
5. WINDOW	Grouping keys and aggregates. Windows are computed here.
6. QUALIFY	Window results. The first step that can see them.
7. SELECT	Everything above. Aliases come into existence here.
8-10. DISTINCT, ORDER BY, LIMIT	The projected output, in that order.

THE TAKEAWAY

Read it as a one-way street. A step can refer to everything above it and nothing below it. There is no exception in this table.

One query, all ten steps

The same query you would write on a Tuesday, numbered by when each line runs.

```
1 SELECT  s.region_key,           -- 7
2         SUM(f.net_amount) AS sales -- 3
3 FROM    fact_sales_line f      -- 1
4 JOIN    dim_store s           -- 1
5 ON      s.store_key = f.store_key -- 1
6 WHERE   f.sold_ts >= DATE '2026-08-01' -- 2
7 GROUP   BY s.region_key        -- 3
8 HAVING  SUM(f.net_amount) > 100000 -- 4
9 ORDER   BY sales DESC          -- 9
10 LIMIT  5;                     -- 10
```

Read the comment column top to bottom: 7, 3, 1, 1, 1, 2, 3, 4, 9, 10.

THE TAKEAWAY

The numbers are not in order and the query is completely ordinary. Nothing here is a trick or an edge case.

Logical order is a contract about **meaning**,
not a description of execution.

What the engine is actually free to do

- ◆ **Reorder joins** and push predicates down below them, so less data is ever touched.
- ◆ **Skip work entirely.** A filter that excludes a partition means the partition is never read.
- ◆ **Its one obligation** is that the answer matches what the logical order specifies.
- ◆ **So never argue about speed from this list.** It describes meaning. The query plan describes cost.

THE TAKEAWAY

Two different questions, two different tools. Logical order answers 'what does this mean'. The plan answers 'why is it slow'. Module twenty one is the second one.

WHERE runs after the join. Which is why **a filter can undo an outer join.**

Step one is bigger than it looks

- ◆ **FROM produces a row multiset**, not a table. Duplicates are expected and order means nothing yet.
- ◆ **Joins inside step one are themselves ordered**, left to right, unless parentheses say otherwise.
- ◆ **LATERAL is the one exception in the whole list**. It lets a FROM item see the FROM items written before it.
- ◆ **Everything else at step one is blind**. A plain subquery in FROM cannot reference its siblings, and never could.

THE TAKEAWAY

LATERAL exists precisely because step one is otherwise sealed. When people say SQL has no variables, this is the clause that argues back.

A LEFT JOIN that is not a LEFT JOIN

```
1 SELECT s.store_name,  
2        COUNT(f.sale_key) AS lines_sold  
3 FROM dim_store s  
4 LEFT JOIN fact_sales_line f  
5     ON f.store_key = s.store_key  
6 WHERE f.sold_ts >= DATE '2026-08-01'  
7 GROUP BY s.store_name;
```

- 6 A predicate on the right-hand table, applied after the join. Unmatched rows hold a null here and fail it.
- 4 The LEFT JOIN is still written, and it no longer does anything. This query is an inner join.

THE TAKEAWAY

No error, no warning, and a result that looks entirely reasonable until somebody counts the stores.

Where the predicate belongs

IN THE WHERE CLAUSE

Filter after joining

```
1  LEFT JOIN fact_sales_line f
2    ON f.store_key = s.store_key
3  WHERE f.sold_ts >= DATE '2026-08-01'
```

Applied to the joined result. Null rows fail the test, and the outer join is undone.

IN THE ON CLAUSE

Filter while joining

```
1  LEFT JOIN fact_sales_line f
2    ON f.store_key = s.store_key
3  AND f.sold_ts >= DATE '2026-08-01'
```

Applied while matching. Unmatched stores survive, with a count of zero.

THE TIEBREAKER

On an inner join these two are interchangeable. On an outer join they are different queries with different row counts.

The rule, and the disguise

- ◆ A predicate on the **null-supplying table** belongs in the ON clause.
- ◆ A predicate on the **preserved table** belongs in WHERE. It chooses which rows you start with.
- ◆ On an **INNER JOIN** it makes no difference. Which is why the habit forms in the first place.
- ◆ **IS NOT NULL** on the **right-hand table** is the same bug wearing a disguise, and it looks deliberate.

THE TAKEAWAY

Ask which side of the join the column comes from. That one question decides where every predicate goes.

Anti-join, or accident

THE ACCIDENT

A filter that undoes the join

- LEFT JOIN, then WHERE f.sold_ts >= ...
- Tests a real value against nulls
- Silently becomes an inner join
- The author wanted every store

THE PATTERN

A deliberate anti-join

- LEFT JOIN, then WHERE f.sale_key IS NULL
- Tests for the absence of a match
- Keeps exactly the unmatched rows
- The author wanted only the gaps

THE TIEBREAKER

IS NULL on the join key is the anti-join. Any other predicate on that table is the bug. The two are one keystroke apart and they read almost identically.

The anti-join, two ways

OUTER JOIN AND IS NULL

Explicit and old

```
1 SELECT s.store_key
2   FROM dim_store s
3  LEFT JOIN fact_sales_line f
4    ON f.store_key = s.store_key
5     AND f.sold_ts >= DATE '2026-08-01'
6  WHERE f.sale_key IS NULL;
```

The date predicate must be in ON. In WHERE it would eliminate the very rows this query is looking for.

NOT EXISTS

Says what it means

```
1 SELECT s.store_key
2   FROM dim_store s
3  WHERE NOT EXISTS (
4      SELECT 1
5        FROM fact_sales_line f
6       WHERE f.store_key = s.store_key
7             AND f.sold_ts >= DATE '2026-08-01')
```

One place for every predicate, and no way to put one in the wrong clause.

THE TIEBREAKER

Both are correct and most optimisers plan them identically. NOT EXISTS is preferred because it removes the opportunity to make the mistake.

Kestrel's weekly store coverage report lists all 412 trading stores with the number of sales lines each recorded. It is built with a LEFT JOIN from dim_store to fact_sales_line, and a WHERE clause restricting to the reporting week.

For the week of the August refurbishment programme, the report shows 394 stores. Operations asks where the other eighteen went. All eighteen were closed for refurbishment for the whole week, and every one of them is flagged as trading in dim_store.

Why are eighteen stores missing?

- ☐ A The join direction is wrong. It should be a RIGHT JOIN from the fact table to the dimension.
- ☐ B The week predicate is in WHERE, so it runs after the join and rejects the null-filled rows.
- ☐ C Refurbishing stores are excluded upstream, because the trading flag is cleared during a closure.
- ☐ D COUNT returns zero for those stores and the reporting layer suppresses zero-valued rows.

◆ PAUSE AND DECIDE

Why are eighteen stores missing?

A Wrong join direction

Reverses which side is preserved. A right join from the fact table protects sales lines, and sales lines were never at risk.

C Trading flag cleared upstream

The best distractor here, and the scenario rules it out in the last clause. All eighteen are flagged as trading.

B WHERE runs after the join

An unmatched store carries a null timestamp. Comparing null to a date gives unknown, and the row is discarded at step two.

D Zero rows suppressed downstream

COUNT of a null column correctly returns zero. Those rows never reach COUNT, because they were eliminated three steps earlier.

THE SIGNAL

"Closed for the whole week." A store with no matching sales lines is exactly the row an outer join exists to preserve, and exactly the row a WHERE predicate on the fact table deletes.

SELECT is evaluated seventh. Its aliases **do not exist** until then.

Why WHERE cannot see your alias

```
1 SELECT product_key,  
2       net_amount - discount_amount AS net_after_disc,  
3       net_amount / NULLIF(qty, 0)   AS unit_price  
4 FROM fact_sales_line  
5 WHERE net_after_disc > 0;
```

- 5 WHERE is step two. The alias on line 2 comes into existence at step seven, five steps later.
- 2 Repeating the whole expression in WHERE works in every dialect. It is verbose and it is correct.

THE TAKEAWAY

The engine reports an unknown column. It is not unknown. It is not yet computed, and the difference is the whole lesson.

Where a SELECT alias is visible

CLAUSE	CAN IT SEE A SELECT ALIAS?
WHERE	No. Not in any major dialect, and that will not change.
GROUP BY	Postgres, Redshift, MySQL and Snowflake yes. SQL Server no.
HAVING	MySQL and Snowflake yes. Postgres, Redshift and SQL Server no.
QUALIFY	Yes, wherever QUALIFY exists at all.
ORDER BY	Yes, everywhere. The one case the standard has always required.

THE TAKEAWAY

ORDER BY is the only row you can rely on. Everything between WHERE and QUALIFY is a portability decision you are making whether you notice or not.

A wart, in the words of the man who maintains it

- ◆ **Postgres allows a bare alias in GROUP BY** as an extension, and has since long before anyone reading this started.
- ◆ **But only a bare alias.** Put it inside an expression and the name is read as an input column again.
- ◆ **Tom Lane calls it a wart**, because it conflicts with the logical model of query evaluation.
- ◆ **Which is the useful part.** The exception is explicitly an exception, so the model still holds.

THE TAKEAWAY

An extension that works for a bare name and stops working the moment you wrap it is worse than no extension. Know it exists so it does not surprise you.

Two ways to reuse an expression

REPEAT IT

Correct, and it drifts

```
1 SELECT net_amount - discount_amount
2       AS net_after_disc
3 FROM fact_sales_line
4 WHERE net_amount - discount_amount > 0;
```

Two copies of one rule. The day somebody edits one of them, the query is quietly wrong.

NAME IT ONCE

A CTE creates a scope

```
1 WITH priced AS (
2   SELECT sale_key,
3          net_amount - discount_amount
4          AS net_after_disc
5   FROM fact_sales_line
6 )
7 SELECT * FROM priced
8 WHERE net_after_disc > 0;
```

By the time the outer WHERE runs, the alias is a real column of a real query block.

THE TIEBREAKER

This is why analytics SQL is written in CTEs. Not for tidiness. Because a CTE creates a scope in which your names actually exist.

GROUP BY does not filter rows. It **replaces them.**

After GROUP BY, the detail is gone

- ◆ **One row per distinct combination** of grouping keys. Nothing else survives as a row.
- ◆ **Only two things are addressable** after that: the grouping keys, and aggregates over what was collapsed.
- ◆ **A bare non-key column has no single value to return**, because it had many. Hence the error.
- ◆ **Postgres and MySQL relax this** when you group by a primary key, since every other column is then determined.

THE TAKEAWAY

The message says a column must appear in the GROUP BY clause. What it means is that the column no longer has one value.

WHERE or HAVING

WHERE

Before grouping

- Filters detail rows
- Runs at step two
- Cannot see aggregates
- Reduces what has to be grouped

HAVING

After grouping

- Filters whole groups
- Runs at step four
- Can see aggregates
- Everything was grouped first

THE TIEBREAKER

If a predicate does not mention an aggregate, it belongs in WHERE. Putting it in HAVING is usually still correct, and always does more work.

The same threshold, two different meanings

```
1 SELECT store_key,  
2        SUM(net_amount) AS sales  
3 FROM fact_sales_line  
4 WHERE promo_key IS NOT NULL  
5 GROUP BY store_key  
6 HAVING SUM(net_amount) > 100000;
```

- 4 Filters lines before grouping, so the total is promotional sales only.
- 6 Filters stores after grouping. Neither predicate can be moved to the other clause.

THE TAKEAWAY

Sixty two percent of Kestrel's sales lines carry no promotion. That WHERE clause removes nearly two thirds of the input before a single group is formed.

GROUP BY ALL, and grouping by number

- ◆ **GROUP BY ALL** groups by every non-aggregated item in the SELECT list. Snowflake, Databricks and DuckDB have it.
- ◆ **GROUP BY 1, 2** groups by ordinal position in the SELECT list, and has worked almost everywhere for decades.
- ◆ **Both read the SELECT list at step three**, which the list on the earlier slide says has not run. They are parser conveniences, not a change to the order.
- ◆ **Ordinals break silently under edits.** Insert a column at position two and every number after it now means something else.

THE TAKEAWAY

Convenience at authoring time, ambiguity at review time. Neither is wrong, and neither belongs in a query somebody else will maintain.

HAVING with no GROUP BY

- ◆ **A query with an aggregate and no GROUP BY still groups.** The whole result becomes one group.
- ◆ **So HAVING is legal without GROUP BY,** and filters that single group in or out entirely.
- ◆ **It returns one row or no rows.** Never the detail rows, however tempting that reading is.
- ◆ **Useful for a guard.** Return the total only when the row count clears a disclosure threshold.

THE TAKEAWAY

It surprises people because it looks like a WHERE clause that forgot its grouping. It is not. Step three ran, with one group.

An analyst reports the number of Kestrel stores that took more than one hundred thousand dollars in promotional sales last week. The query filters `promo_key` IS NOT NULL in the WHERE clause, groups by store, and applies the threshold in HAVING.

A colleague writes what they believe is the same report, but moves the promotion filter into HAVING as `COUNT(promo_key) > 0`. The two reports disagree by forty one stores. Both analysts are confident their query is correct, and both queries run without error.

Which query answers the question as stated?

- ☐ A The first. WHERE removes non-promotional lines before any total is formed.
- ☐ B The second. HAVING is the correct clause for any condition applied to a group.
- ☐ C Neither. Both need COUNT DISTINCT, because one transaction contributes many lines.
- ☐ D Both. They are logically equivalent, so the gap must come from a data refresh.

◆ PAUSE AND DECIDE

Which query answers the question as stated?

- A

The first query

The threshold is on promotional sales. Only removing the other lines first produces a total that means that.
- B

The second query

It sums every line, then keeps stores that had at least one promotion. The threshold ends up measured on total sales.
- C

Both need COUNT DISTINCT

Fixes a grain problem that is not present. The measure is a sum of amounts, and lines are the declared grain.
- D

They are equivalent

The forty one store gap is the proof that they are not. Equivalence gets demonstrated, never assumed, which is module twenty.

THE SIGNAL

"More than one hundred thousand dollars in promotional sales." The threshold is on a filtered total, so the filter has to run before the total is formed.

Window functions run **after** WHERE and GROUP BY, and **before** SELECT.

What that position implies

- ◆ It sees the rows that survived **WHERE**, and only those. Filtering changes every window result.
- ◆ It cannot be named in **WHERE**, **GROUP BY** or **HAVING**, because all three are evaluated earlier.
- ◆ It can be named in **QUALIFY**, in **ORDER BY**, and in an **outer query**. Those are the three options.
- ◆ It can take an aggregate as its argument, because grouping has already finished at step three.

THE TAKEAWAY

Everything surprising about window functions is downstream of one position in a ten-step list. Learn the position, and the behaviour stops needing memorising.

Change the filter, change every window

- ◆ **The window sees post-WHERE rows.** Add a predicate and every running total, rank and lag in the query moves.
- ◆ **Filtering to one store does not give you that store's rank nationally.** It gives you a rank of one.
- ◆ **Filtering to August does not give an August-to-date balance** carried forward. It restarts the running sum at zero.
- ◆ **If you need the wider context, compute the window first**, in a CTE, and filter the result afterwards.

THE TAKEAWAY

'Rank within what?' is the question to ask of every window function. The answer is always the rows that survived step two, and never the table.

Filtering a window function in WHERE

```
1 SELECT store_key,  
2        product_key,  
3        net_amount,  
4        ROW_NUMBER() OVER (  
5          PARTITION BY store_key  
6          ORDER BY net_amount DESC  
7        ) AS rn  
8 FROM fact_sales_line  
9 WHERE rn <= 10;
```

9 WHERE is step two. The window function above it is step five, and its alias is step seven.

THE TAKEAWAY

This is the alias error and the window error at once. Two reasons the same line cannot work, and neither of them is about ROW_NUMBER.

Three legal places to filter it

- ◆ **QUALIFY**, where the engine has it. One flat query, no nesting, reads like HAVING for windows.
- ◆ **A CTE or subquery**, then filter in the outer WHERE. Works in every dialect ever written.
- ◆ **ORDER BY with LIMIT**, when you want the top of the whole result rather than the top of each group.
- ◆ **Not WHERE, not GROUP BY, not HAVING**. No dialect permits it and none is going to.

THE TAKEAWAY

QUALIFY is the flat version and the CTE is the portable version. Module eight is entirely about choosing between them.

QUALIFY, by engine

ENGINE	QUALIFY
Amazon Redshift	Yes, since July 2023.
Snowflake	Yes. The clause originates with Teradata.
Databricks SQL	Yes.
BigQuery and DuckDB	Yes. DuckDB treats it as a first-class clause.
PostgreSQL and SQL Server	No. Wrap the query instead.

THE TAKEAWAY

Kestrel's warehouse is Redshift and its store-ops dashboard runs on Postgres. The same logic gets written twice, and that is normal rather than a failure.

The named WINDOW clause

Step five has a keyword of its own, and almost nobody uses it.

```
1 SELECT store_key, sold_ts, net_amount,  
2        SUM(net_amount) OVER w AS running,  
3        AVG(net_amount) OVER w AS mean_so_far,  
4        COUNT(*) OVER w AS n_so_far  
5 FROM fact_sales_line  
6 WHERE store_key = 41  
7 WINDOW w AS (ORDER BY sold_ts, sale_key  
8              ROWS UNBOUNDED PRECEDING)  
9 ORDER BY sold_ts;
```

7 Defined once, used three times. The definition is written below the SELECT list and evaluated before it.

8 Three windows that must agree can now only be changed together, which is the real benefit.

THE TAKEAWAY

The repetition is not just verbose. Three copies of a frame definition is three chances for one of them to drift.

A window over an aggregate

```
1 SELECT s.region_key,  
2        SUM(f.net_amount) AS region_sales,  
3        SUM(SUM(f.net_amount)) OVER () AS total_sales,  
4        SUM(f.net_amount)  
5        / SUM(SUM(f.net_amount)) OVER () AS share  
6 FROM fact_sales_line f  
7 JOIN dim_store s ON s.store_key = f.store_key  
8 GROUP BY s.region_key;
```

- 3 The inner SUM aggregates within the group at step three. The outer SUM is a window over the grouped result at step five.
- 5 Percent of total in a single pass. No self-join, no second scan, no subquery.

THE TAKEAWAY

A doubled aggregate looks like a typo and is not. It is only legal because grouping finishes two steps before windows begin.

Reading a doubled aggregate

- ◆ **Read it inside out.** Inner call at step three, one value per group. Outer call at step five.
- ◆ **The window operates on seven rows,** not on 2.41 billion. It runs after the collapse, not before.
- ◆ **An empty OVER means the whole result.** Add PARTITION BY to get a subtotal instead.
- ◆ **Nesting an aggregate inside an aggregate without OVER** is a genuine error, and always was.

THE TAKEAWAY

Once you can read this shape, percent of total, ratio to report and index against average are all the same query with a different divisor.

Windows cannot nest

- ◆ **A window function cannot be an argument to another window function.** One step, one pass.
- ◆ **So a running total of a moving average** is two query blocks, not one expression.
- ◆ **An aggregate inside a window is fine**, because the aggregate belongs to step three and the window to step five.
- ◆ **The asymmetry is the tell.** Different steps can nest. The same step cannot nest inside itself.

THE TAKEAWAY

Stack the CTEs. One window per level, and each level gets the previous one's output as ordinary columns.

A merchandiser wants the three best-selling products in each of Kestrel's seven regions by net sales. They write a query computing a RANK over products within region, and add a WHERE clause keeping ranks of three or less. Redshift rejects it.

They rewrite it with the rank filter moved into HAVING, and Redshift rejects it again with a similar message. They open a ticket asking whether Redshift supports ranking functions at all, and note that the same query works in a spreadsheet.

What should they be told?

- A Add a GROUP BY on region and product, so that the ranking has defined groups to rank within.
- B Use ROW_NUMBER rather than RANK. RANK cannot be filtered on because it can produce ties.
- C The rank is computed at step five and both clauses run earlier. Use QUALIFY, or wrap the query.
- D Compute the ranks in a separate pass and join the result back to the fact table on region and product.

◆ PAUSE AND DECIDE

What should they be told?

- A

Add a GROUP BY

Adds a grouping the query never needed, and does not touch the error. The rank still cannot be seen from step two.
- B

Swap RANK for ROW_NUMBER

Both sit at step five and neither can be filtered from WHERE. Ties are a real issue and they belong to module four.
- C

Wrong step, not a missing feature

Redshift has both ranking and QUALIFY. One keyword fixes it, and a CTE is the version that also runs on the Postgres dashboard.
- D

A second pass and a join back

Correct, and it buys a second scan of a 2.41 billion row table in order to avoid typing one keyword.

THE SIGNAL

"Rejects it, and rejects it again." Two different clauses, one error, because both run before the window step. The engine is not missing a feature.

DISTINCT runs after SELECT. So it deduplicates **what you projected**, not what you meant.

DISTINCT cannot rescue a window function

```
1 SELECT DISTINCT
2     customer_key,
3     ROW_NUMBER() OVER (ORDER BY sold_ts) AS rn
4 FROM fact_sales_line
5 WHERE store_key = 41;
```

- 3 Row number is unique on every row by construction, so no two projected rows can ever match.
- 1 DISTINCT removes nothing, and still pays for the sort. The query is slower and no more correct.

THE TAKEAWAY

Project any column that is unique per row and DISTINCT becomes a no-op that still costs a full sort. It is the most expensive way to write nothing.

ORDER BY and LIMIT, the last two steps

- ◆ **ORDER BY sees the projected output**, which is why aliases work there in every dialect.
- ◆ **LIMIT is applied last**. Without a total order, which rows you get back is simply undefined.
- ◆ **txn_id is unique per store per day at Kestrel**, so ordering by it alone is not a total order.
- ◆ **Add tiebreaker columns until the ordering is unique**. Then the result is reproducible, by construction.

THE TAKEAWAY

A query with LIMIT and a non-unique ORDER BY does not have a single correct answer. It has a set of permitted answers, and re-running it picks again.

ORDER BY inside a CTE is not a promise

- ◆ **A CTE or subquery produces a relation**, and a relation has no order. The standard makes no guarantee here.
- ◆ **It often appears to work**, which is worse, because the query passes review and then a plan change breaks it.
- ◆ **Some engines reject it outright**. Others accept it and discard it during optimisation without saying so.
- ◆ **Order in the outermost query only**. If an inner order matters, encode it as a column with ROW_NUMBER.

THE TAKEAWAY

Ordering is a property of a result set, not of a relation. Only the final SELECT produces a result set, so only the final ORDER BY is binding.

A nightly extract sends the five hundred largest sales lines from the previous day to a fraud review queue. It orders by `net_amount` descending and applies `LIMIT 500`. The query has run unchanged for two years.

After a cluster resize, the review team reports that roughly thirty records differ each night from what the previous configuration produced, including on days re-run against frozen, identical data. The row count is always exactly five hundred and the total value barely moves.

What is the defect?

- ☐ A The resize changed data distribution, so some rows are now genuinely absent from the extract.
- ☐ B The extract should order ascending and take the last five hundred, which is stable across distributions.
- ☐ C `LIMIT` should be replaced by `ROW_NUMBER` with a filter, which is deterministic where `LIMIT` is not.
- ☐ D `net_amount` is not unique, so rows tied at the boundary are chosen arbitrarily and the choice is unstable.

What is the defect?

A Rows genuinely absent

Nothing is absent. Exactly five hundred arrive every night and the total barely moves, which is the shape of substitution, not loss.

C ROW_NUMBER instead of LIMIT

ROW_NUMBER over the same non-unique ordering is exactly as arbitrary. Determinism comes from the ORDER BY, never from the function.

B Order ascending instead

Reverses the direction and keeps the tie. The boundary is still undefined, and now the query reads backwards for no gain.

D An incomplete ordering

Many lines share an amount. The ordering is partial, so any of the tied rows may occupy the last places, and the plan decides.

THE SIGNAL

"Identical data, different rows." That is the signature of an incomplete ordering, and no amount of re-running will settle it. Add a tiebreaker until the order is unique.

Reading somebody else's query

- ◆ **Find the FROM first, not the SELECT.** You cannot judge anything until you know what is being joined to what.
- ◆ **Then the WHERE, and ask which table each predicate belongs to.** That one pass finds the outer-join bug.
- ◆ **Then the grain.** What does one row represent after step three, and does the SELECT list agree?
- ◆ **Read the SELECT list last,** which is when the engine reads it. It is the output, not the logic.

THE TAKEAWAY

Reading a query in written order is reading it in the one order that does not explain it. Reading it in evaluation order takes the same time and finds more.

Diagnosing any 'why can't I do that'

01

Name it

Which clause holds the error, and which step number is that clause?

02

Locate it

Which step creates the thing you referred to?

03

Compare

If the second number is larger, that is the entire explanation.

04

Move

Either refer to it later, or compute it earlier. There is no third option.

05

Choose

Later means QUALIFY or ORDER BY. Earlier means a CTE or a subquery.

THE TAKEAWAY

Four of the five failures in this module resolve in under ten seconds with this. The fifth, the outer join, produces no error at all, which is why it gets its own habit.

Error message to cause

WHAT THE ENGINE SAYS	WHAT IT ACTUALLY MEANS
Column 'rn' does not exist	You named a window result before step five had run.
Column must appear in the GROUP BY clause	After grouping, that column no longer has one value.
Aggregate functions are not allowed in WHERE	Aggregates form at step three. WHERE is step two.
Window functions are not allowed in WHERE	The same sentence, two steps later.
No error. Rows are simply missing.	An outer join was undone by a WHERE predicate.

THE TAKEAWAY

Four of these stop the query, which makes them cheap. The last one runs, passes review, and gets published. That asymmetry is the argument for module twenty.

Three habits that make this a non-issue

- ◆ **Stack CTEs, one idea per level.** Every name you create becomes a real column for the level above.
- ◆ **Predicates on the null-supplying side of an outer join go in ON.** Every time, without thinking about it.
- ◆ **Every ORDER BY that feeds a LIMIT ends with a unique column.** Reproducibility becomes structural.
- ◆ **None of these is about knowing the ten steps.** They are about not needing to recall them under pressure.

THE TAKEAWAY

Understanding the order stops you being surprised. These three habits stop the situation arising, which is better.

The ten steps tell you what a query **means**.
Module two asks whether it means
anything at all.